How Airbnb safeguards changes in production

medium.com/airbnb-engineering/how-airbnb-safeguards-changes-in-production-c83e94bfc52

Zack Loebel-Begelman September 6, 2022

Creative engineers and data scientists building a world where you can belong anywhere. http://airbnb.io

Part II: Near Real-time Experiments

Press enter or click to view image in full size

In our first post we discussed the need for a near real time Safe Deploy system and some of the statistics that power its decisions. In this post we will cover the architecture and engineering choices behind the various components that Safe Deploys comprises.

Designing a near real-time experimentation system required making explicit tradeoffs among speed, precision, cost, and resiliency. An early decision was to limit near real-time results to only the first 24 hours of an experiment — enough time to catch any major issues and transition to using comprehensive results from the batch pipeline. The idea being once batch

results were available, experimenters would no longer need real time results. The following sections describe the additional design decisions in each component of the Safe Deploys system.

High Level Design

There are 3 major components that make up the technical footprint of the Safe Deploys system:

1. , a Flink job that acts as a centralized coordinator, providing experiment configuration to NRT via Kafka and invoking statistical computations by calling Measured via HTTP.
2. , another Flink job that extracts measures, joins and enriches those measures with assignment information (treatment and subject information), and stores the enriched measures into S3.
3. , a python library (invoked via a Python HTTP server and worker pool) that consumes enriched measures from S3, aggregates them, and runs stats to determine if any change is significant.

Press enter or click to view image in full size

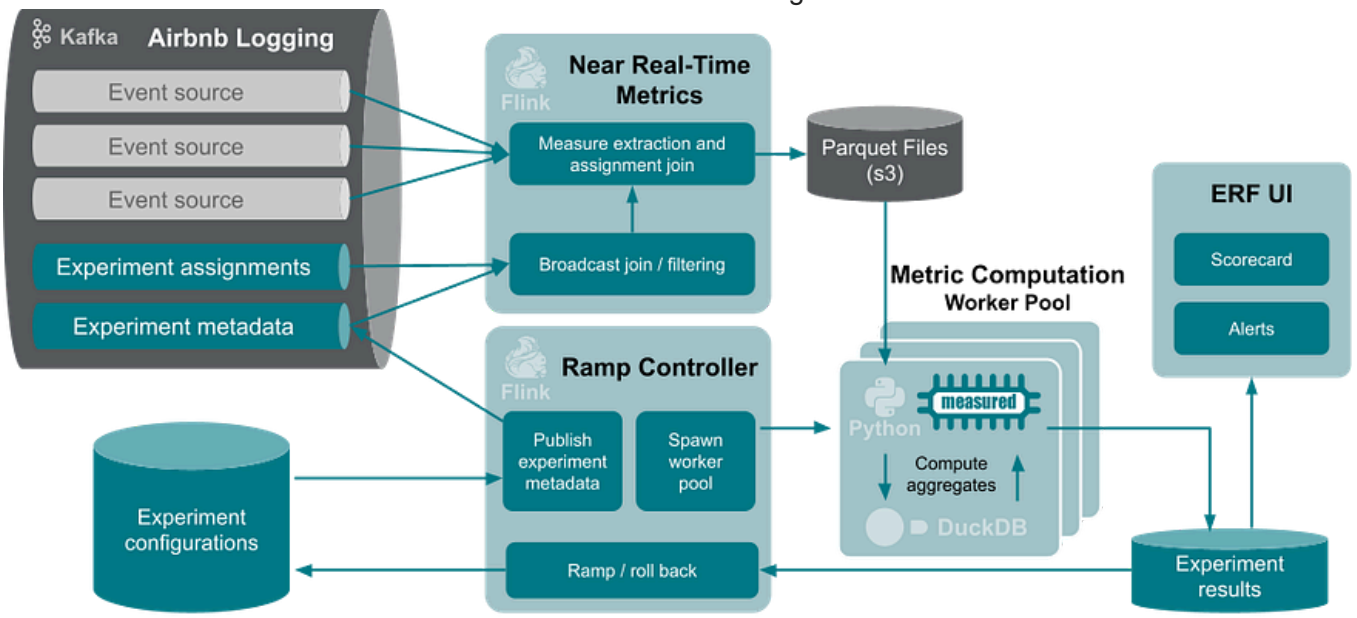

Fig 1: Architecture Diagram of the Safe Deploy system

Ramp Controller

The Ramp Controller performs automated experiment ramping based on the results from Measured. It increases experiment exposure in stages, slowly exposing more subjects and monitoring metric impacts at each stage. If any egregiously negative metric is observed, the Ramp Controller will immediately shut down the experiment to minimize the impacts of bad changes. It supports several ramping algorithms, but most users leverage a simple time based algorithm.

Press enter or click to view image in full size

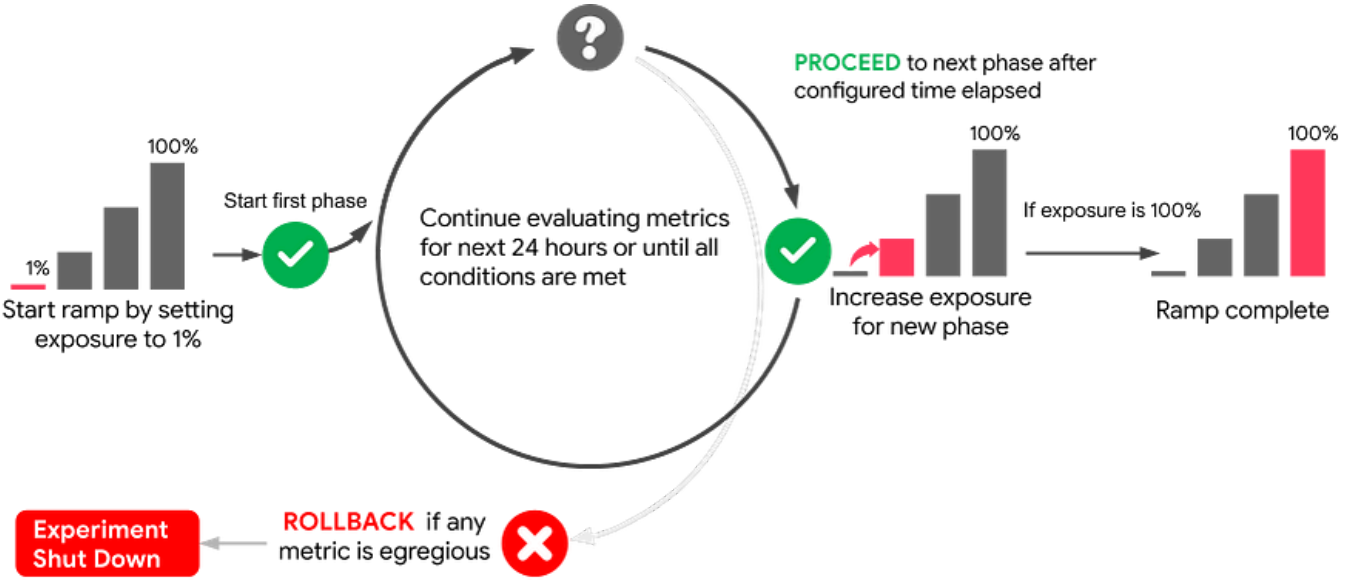

Figure 2: Ramping process

Ramp Controller was designed to be stateless, and resilient to any job failures. Within seconds of an experiment starting, it publishes metadata to Kafka, triggering NRT to start joining events for that experiment. The metadata includes a path in S3 that NRT will write to. At this point the Ramp Controller's core loop will begin:

Press enter or click to view image in full size

Ramp Controller core loop	Measured job
1. Invoke a Measured job providing the S3 path to read from	
2. Poll Measured job checking for results	Run stats for all the events at the S3 path at that time
3. Evaluate the results and make a decision to ramp, rollback, or continue waiting	
4. Wait some amount of time (2 minutes for the first 6 hours, and 30 minutes for hours 7-24), and then invoke another job (Step 1)	

Results are computed for the first 24 hours of the experiment, with new metrics consumed as new files are published to S3. A metric is marked egregious when the percent change is smaller than -20% with an adjusted p-value of less than or equal to 0.01. By leveraging the Measured framework for metric computation, we get custom aggregations, richer statistical models, and the ability to compute performance metrics, and dimensional cuts for metrics.

After overcoming these technical challenges in scaling the pipeline and tuning decision making, we were ready to vet the system with experiment owners and drive adoption.

Near Real Time (NRT) Pipeline

We built the new NRT pipeline in Java and Scala using Apache Flink. It reads from a multitude of Kafka streams: event streams containing raw user based events (impressions, booking requests etc.), a stream that the Ramp Controller emits containing experiment metadata, and the streaming that contains the raw assignment events emitted for all experiments which are also consumed by the batch pipeline.

Previously Airbnb had attempted to build an online data store for all experiment assignments, however this did not scale and was eventually shut down. By reducing the scope, specifically limiting the NRT pipeline to the first 24 hours of an experiment, we are able to store a bounded subset of assignments. Using a broadcast join of experiment metadata lets us filter the assignment events and Flink makes aging out data trivial.

The extraction is written in a stand-alone library so that the measure definitions can be re-used in both batch and streaming. In order to be highly performant, the measure extraction determines which events to extract first using an inverted index based on the existence and values of json fields then only running extraction on the relevant events. Not only do we extract measures, but also dimensions from each event. Because we want to limit the complexity in this job, we only support dimensions from the same event as a measure itself.

Our first difficulty was in how to handle measures and assignments coming in out of order. We want data to age out at different times when joining assignment events to measures and assignment data should be stored for the full 24 hours. Because the volume of measure events means we can't keep them for 24 hours, we keep a short buffer dropping measures after 5 minutes. The outer join required to achieve this goal required building a custom join using the keyed co-process api.

Once the data is joined we buffer it internally within Flink to reduce the total number of files for small experiments. We wrote a simple keyed process stage that hashes the events based on the timestamp against how many concurrent files we want to output. It's important that we hash on the timestamp since Flink requires the keying mechanism to be deterministic. The events are buffered based on event counts and time, emitting the buffered list either once a partition hits an event based or time based threshold. This stage allows us to have more fine grained control over the number of files we output.

Join Medium for free to get updates from this writer.

Remember me for faster sign in

We leverage Flink's built in support for parquet and S3 as a file sink to write the files. In order to provide exactly-once semantics, Flink will only write files when checkpointing occurs. Files output by the NRT pipeline are consumed by the ramp controller to make decisions. To keep our latency low, we checkpoint every 5 minutes.

Measured

Measured is a framework for defining and computing metrics. It consists of a Scala library for extracting measures and dimensions from raw events that the NRT pipeline leverages, and a Python library for defining metrics (based off of those measures), statistical models, and visualizations. This section focuses on the Python library, and how it is used to compute metrics.

In order to provide consistent results across platforms we run the same Measured jobs that user's run via a Python HTTP job server and worker pool. The NRT metric evaluation is one of those jobs, it downloads the event files from S3 using a Python worker pool. Once the files are downloaded the job leverages duckdb's parquet reader functionality to aggregate to the user level. Once we have a local user aggregate the job evaluates the various sequential models discussed in the first post. The results of these evaluations are stored in a MySQL database upon job completion to be retrieved by the UI or the Ramp Controller over HTTP.

Adoption

The full vision of Safe Deploys encompassed safeguarding any changes in production. However, to gain experience and trust, we initially focused our efforts on A/B tests. We knew that Safe Deploys, like any anomaly detection system, especially one that automates remediation steps, would face certain challenges in adoption, including:

- Trust in NRT metrics that were similar but not exactly the same as existing batch ones
- Relinquishment of control in ramping and shutdown of experiments
- False positives that could slow down experimentation by forcing restarts

Before Safe Deploys, nearly a quarter of Airbnb teams had a manual process for ramping up experiments. This consisted of increasing the exposure of an experiment, manually verifying performance metrics, and repeating until reaching a target exposure. This often masked significant but not visually obvious negative impacts.

Press enter or click to view image in full size

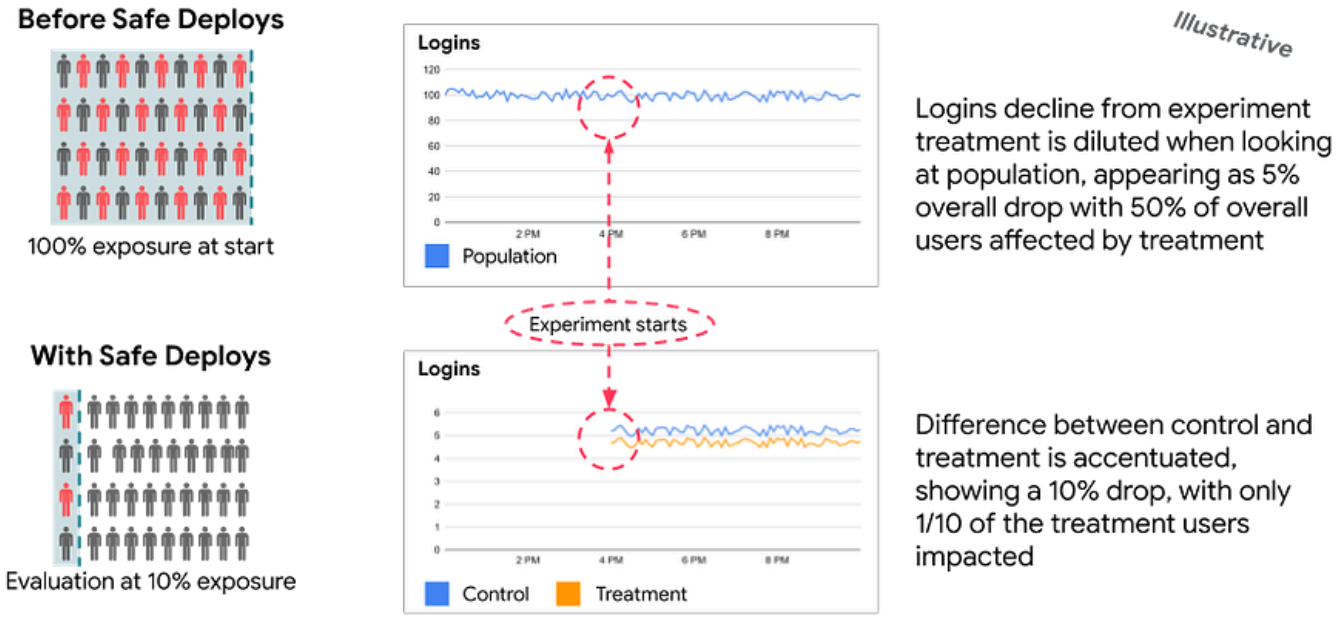

Figure 3: Illustration of how a controlled experiment provides greater sensitivity

We evangelized Safe Deploys as complementary to the existing process, providing increased sensitivity of detecting negative impacts, while still allowing experimenters to stop an experiment based on their own monitoring at any time. We also continually improved statistical methods used to decrease false positives and negatives. Since enabling Safe Deploys by default a year ago, it has been used for over 85% of experiment starts and helped prevent dozens of incidents, and flagged misconfigurations early, minimizing negative impacts to Airbnb's business and wasted engineering resources on remediation.

Tip of the Iceberg

Safeguarding experiments was a significant step towards reducing incidents at Airbnb, however the full vision encompasses changes originating from other channels. The distribution of changes across different channels can be found in Figure 4.

Press enter or click to view image in full size

Figure 4: Distribution of changes pushed to production by channel

We tackled each remaining channel differently:

- Feature Flags were unified with Experiments to gain Safe Deploys capabilities
- Content management systems were provided APIs to programmatically create experiments tied to content changes, and ramped with Safe Deploys

- Code Deploys through Spinnaker ran Safe Deploys alongside pre-existing Automated Canary Analysis with each deploy

(We considered infrastructure configs out of scope, since these lower level changes would require a fundamentally different approach to address.)

Code Deploys with Spinnaker account for an outsized majority of changes in production and required extensive work to enable. The next post in this series will cover how we achieved this through changes across traffic routing, service configs, and dynamically created configurations for Spinnaker.

Interested in working at Airbnb? Check out our open roles.

Acknowledgements

Safe Deploys was only made possible through the combined efforts across Airbnb's infrastructure and data science teams. We would like to thank Jingwei Lu, Wei Ho, and the rest of the Stream Infrastructure team for helping implement, and subsequently scale the NRT pipeline. Also, thanks to Candace Zhang, Erik Iverson, Minyong Lee, Reid Andersen, Shant Toronsean, Tatiana Xifara, and the many data scientists that helped build out metrics and verify their correctness. Also thanks to Kate Odnus, Kedar Bellare and Phil MacCart, who were early adopters and provided us invaluable feedback. In addition Adam Kocoloski, Raymie Stata and Ronny Kohavi for championing the effort across the company. We would also like to thank other members of the ERF team that contributed to Safe Deploys: Adrian Kuhn, Antoine Creux, George Li, Krishna Bhupatiraju, Shao Xie, and Vincent Chan.

> All product names, logos, and brands are property of their respective owners. All company, product and service names used in this website are for identification purposes only. Use of these names, logos, and brands does not imply endorsement.